DATA ANALYTICS PROJECT REPORT

E-Commerce Return Logistics Performance and Cost Optimization

Student Name	Deepa K Raj
Course	Data Analytics Certification
Submission Date	12th Oct 2026

Abstract

E-commerce businesses face challenges in managing product returns, which can affect operational efficiency, shipping expenses, and profitability. This project, titled “E-Commerce Return Logistics Performance and Cost Optimization,” analyzes e-commerce return data to identify return trends, evaluate available cost-related indicators, and recommend potential operational improvements.

The project uses a synthetic Kaggle e-commerce dataset containing 34,500 order records. Python was used for cleaning, feature engineering, and exploratory data analysis; MySQL for structured storage and analytical queries; and Power BI for an interactive four-page dashboard supported by DAX measures.

The analysis identified 1,903 returned orders (5.52%). Fashion recorded a return rate of 8.28%, followed by Electronics at 7.30%. “Not as described” was the most frequently recorded return reason (490 returns; 25.75%). Average recorded shipping cost was 6.63 for returned orders compared with 6.12 for non-returned orders.

The dataset does not separately record reverse-shipping, inspection, or restocking expenses. Accordingly, the study identifies potential cost optimization opportunities rather than verified return-processing costs or financial savings.

1. Introduction

1.1 Project Overview

The growth of e-commerce has increased the importance of effective product-return management. Customers may return products because of incorrect descriptions, defects, changing preferences, delivery issues, or receipt of the wrong item. Reverse logistics concerns the movement and handling of products returning from customers to sellers or warehouses.

This project analyzes return patterns and available shipping-cost and profit-margin indicators using an end-to-end workflow comprising Python, MySQL, and Microsoft Power BI.

1.2 Business Problem Statement

E-commerce businesses need to understand which products are returned most frequently, what reasons customers report, and where operational improvements may be possible. The central question is: How can e-commerce return data be analyzed to identify return patterns and potential opportunities for improving reverse-logistics efficiency and cost management?

1.3 Project Objectives

- Clean and preprocess transaction data using Python.
- Calculate total orders, returned orders, and return rates.
- Analyze returns by product category, region, reason, price, and delivery duration.
- Compare recorded shipping costs and profit-margin values by return status.
- Use MySQL queries to perform structured analysis.
- Build a four-page interactive Power BI report using DAX measures.
- Recommend possible improvements to return management.

1.4 Dataset Description

The synthetic Kaggle dataset contains 34,500 original records and 19 columns. After engineering delivery_days, return_days, and delivery_group, the cleaned dataset contains 22 columns. There are 1,903 returned and 32,597 non-returned orders, giving a return rate of approximately 5.52%.

1.5 Tools and Technologies Used

Tool	Purpose
Python / Pandas	Cleaning, feature engineering, EDA
MySQL	Database storage and SQL analysis
Power BI / DAX	Dashboard, measures, calculated columns
Kaggle	Synthetic dataset source

1.6 Scope of the Project

The study covers return rates, categories, regions, reported return reasons, prices, delivery durations, and recorded shipping-cost and profit-margin values. It does not calculate actual return-processing costs, implement machine-learning prediction, or provide real-time operational monitoring.

Therefore, the project primarily focuses on descriptive and exploratory analysis, providing data-driven insights and identifying possible cost-optimization opportunities rather than measuring actual financial savings.

2. Python: Data Preparation and Exploratory Data Analysis

2.1 Introduction

Python was used to load and inspect the raw dataset, assess data quality, convert date fields, engineer analytical features, and perform exploratory data analysis. The resulting cleaned data was then used for MySQL and Power BI.

2.2 Data Loading and Initial Inspection

The CSV file was loaded using Pandas. The first five rows, dataset dimensions, and initial column data types were inspected. The raw dataset contains 34,500 rows and 19 columns. Date fields initially appeared as text and required conversion during preprocessing.

```
Python 3.13.10 (tags/v3.13.10:4fd8843, Dec  2 2025, 15:08:18) [MSC v.1944 64 bit (AMD
] on win32
Enter "help" below or click "Help" above for more information.
>
======== RESTART: C:\Users\user\Desktop\E-commerce\E-commerce project.py =======
  order_id customer_id product_id  ... profit_margin  customer_age  customer_gender
0  O100000      C17270    P234890  ...         31.17            60           Female
1  O100001      C17603    P228204  ...         -2.62            37             Male
2  O100002      C10860    P213892  ...         13.44            34             Male
3  O100003      C15390    P208689  ...          2.14            21           Female
4  O100004      C15226    P228063  ...          1.15            39             Male

[5 rows x 19 columns]
Dataset Shape: (34500, 19)
order_id               str
customer_id            str
product_id             str
category               str
price              float64
discount           float64
quantity             int64
payment_method         str
order_date             str
delivered_date         str
region                 str
returned               str
request_date           str
return_reason          str
total_amount       float64
shipping_cost      float64
profit_margin      float64
customer_age         int64
customer_gender        str
dtype: object
>
```

Figure 1 — Initial Dataset Inspection Using Python

2.3 Data Quality Assessment

Missing-value and duplicate checks were performed using Pandas. The request_date and return_reason fields each contained 32,597 missing values, corresponding to non-returned orders. These are expected missing values, not errors. All other original columns had no missing values, and the duplicate check identified zero duplicate records.

```
MISSING VALUES
order_id              0
customer_id           0
product_id            0
category              0
price                 0
discount              0
quantity              0
payment_method        0
order_date            0
delivered_date        0
region                0
returned              0
request_date      32597
return_reason     32597
total_amount          0
shipping_cost         0
profit_margin         0
customer_age          0
customer_gender       0
dtype: int64

DUPLICATE RECORDS
0
>>>
```

Figure 2 — Missing Value Assessment and Duplicate Record Detection

2.4 Date Conversion and Preprocessing

The order_date, delivered_date and request_date columns were converted with pd.to_datetime(..., dayfirst=True), enabling date arithmetic. Missing request dates were represented as NaT (Not a Time) after conversion.

```
df['order_date'] = pd.to_datetime(df['order_date'], dayfirst=True)
df['delivered_date'] = pd.to_datetime(df['delivered_date'], dayfirst=True)
df['request_date'] = pd.to_datetime(df['request_date'], dayfirst=True)
```

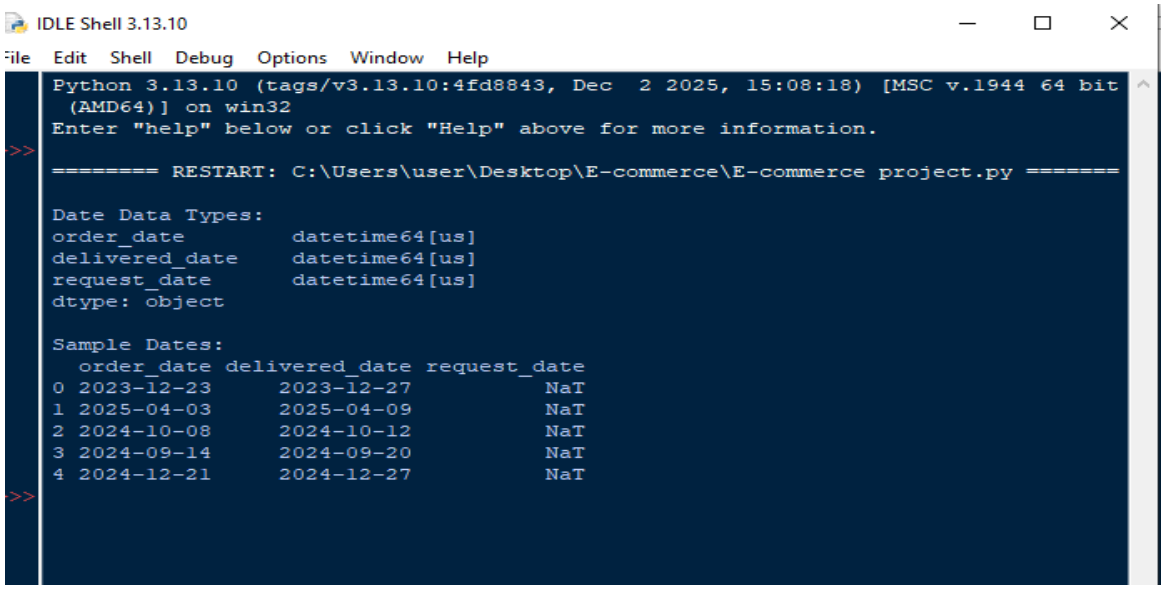


Figure 2.3 — Verified datetime types and converted date samples

2.5 Feature Engineering

Two duration fields were derived from date differences. delivery_days measures the time between order placement and delivery; return_days measures the time between delivery and the return request, where available. A third feature, delivery_group, categorizes delivery duration for comparative analysis.

```
df['delivery_days'] = (df['delivered_date'] - df['order_date']).dt.days
df['return_days'] = (df['request_date'] - df['delivered_date']).dt.days
```

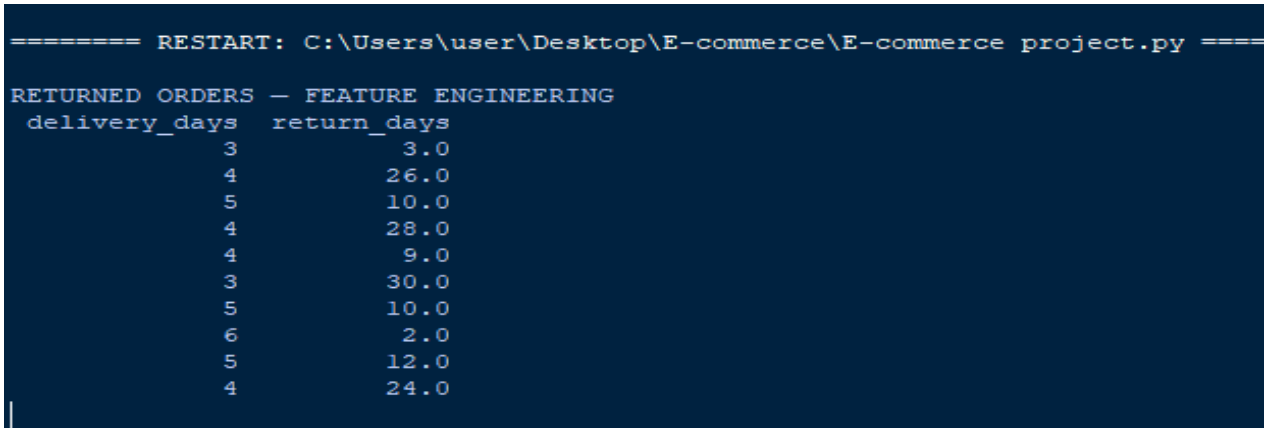


Figure 2.4 — Delivery and return durations for selected returned orders

Delivery groups were formed using pd.cut() with boundaries at 3, 5, 7 and 10 days. The most common group was 4–5 days (20,390 orders), while only two orders were in the 11+ days group. The last group is too small for reliable conclusions about return behavior.

Delivery duration	Orders
1–3 days	4,907
4–5 days	20,390

6–7 days	8,361
8–10 days	840
11+ days	2
Total	34,500

2.6 Exploratory Return Performance Analysis

2.6.1 Overall Return Performance

Among 34,500 orders, 1,903 were marked returned and 32,597 were not returned. The observed return rate was 5.52%, calculated as returned orders divided by total orders, multiplied by 100.

Metric	Value
Total orders	34,500
Returned orders	1,903
Non-returned orders	32,597
Overall return rate	5.52%

2.6.2 Return Rate by Product Category

The original Python analysis grouped records by category, counted orders and returned orders, and calculated the category-level return rate. Fashion had the highest observed rate (8.28%), followed by Electronics (7.30%). Grocery had the lowest (1.31%). These comparisons identify categories for further investigation; they do not establish underlying causes.

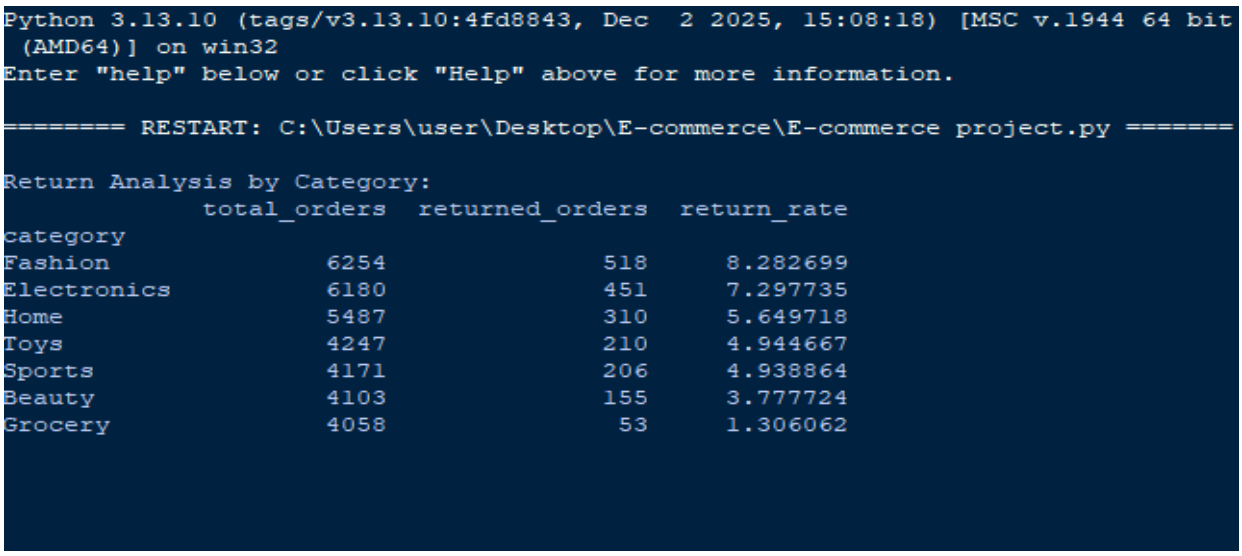


Figure 2.5 — Category-wise order counts and return rates

2.6.3 Return Reasons

For orders marked returned, value_counts() was applied to return_reason. “Not as described” was the most frequently recorded reason (490, 25.75%), closely followed by “No longer needed” (481, 25.28%). These are reported reasons, not verified root causes.

Recorded reason	Returns	Share of returns
Not as described	490	25.75%
No longer needed	481	25.28%
Defective	465	24.44%
Missing/Wrong item	439	23.07%
Slow delivery	28	1.47%

2.6.4 Regional Performance

The East region recorded the highest return rate at 5.91%, followed by South (5.68%), West (5.45%), North (5.36%), and Central (5.10%). The differences were relatively small, so the dataset does not suggest a strong regional separation.

2.7 Shipping Cost and Profit-Margin Analysis

The analysis compared the averages of the dataset’s shipping_cost and profit_margin columns by return status. Recorded shipping cost averaged 6.63 for returned orders and 6.12 for non-returned orders. The profit_margin field averaged 32.48 and 27.86 respectively. The source does not document the profit_margin calculation or units, so these values should not be interpreted as verified post-return net profit.

Recorded metric	Returned	Non-returned
Average shipping_cost	6.63	6.12
Average profit_margin value	32.48	27.86

Importantly, shipping_cost is an order-level recorded field; the comparison does not measure the additional reverse-shipping or return-processing expense. The dataset lacks separate inspection, refund-handling, restocking, and reverse-transport cost fields.

2.9 Consolidated EDA Summary

To summarize the principal exploratory findings in a single reproducible output, a final Python block calculated overall return performance, the highest-return category, the most frequently recorded return reason, and mean recorded shipping costs and product prices by return status. These calculations use Pandas groupby(), value_counts(), and mean() operations.

```
# EDA SUMMARY
# ------------------------------------------

total_orders = len(df)
returned_orders = (df['returned'] == 'Yes').sum()
return_rate = (returned_orders / total_orders) * 100

category_return_rate = (
    df.groupby('category')['returned']
    .apply(lambda x: (x == 'Yes').mean() * 100)
    .sort_values(ascending=False)
)

return_reasons = (
    df[df['returned'] == 'Yes']['return_reason']
    .value_counts()
)

shipping_analysis = (
    df.groupby('returned')['shipping_cost'].mean()
)

price_analysis = (
    df.groupby('returned')['price'].mean()
)

print("\n" + "=" * 50)
print("         EXPLORATORY DATA ANALYSIS SUMMARY")
print("=" * 50)

print("\nOVERALL RETURN PERFORMANCE")
print("Total Orders:", total_orders)
print("Returned Orders:", returned_orders)
print(f"Return Rate: {return_rate:.2f}%")
```

```
print("\nPRODUCT CATEGORY")
print("Highest Return Rate:",
      category_return_rate.index[0],
      f"({category_return_rate.iloc[0]:.2f}%)")

print("\nRETURN REASONS")
print("Most Common Reason:",
      return_reasons.index[0],
      f"({return_reasons.iloc[0]} returns)")

print("\nAVERAGE RECORDED SHIPPING COST")
print(f"Returned Orders: {shipping_analysis['Yes']:.2f}")
print(f"Non-Returned Orders: {shipping_analysis['No']:.2f}")

print("\nAVERAGE PRODUCT PRICE")
print(f"Returned Orders: {price_analysis['Yes']:.2f}")
print(f"Non-Returned Orders: {price_analysis['No']:.2f}")

print("\n" + "=" * 50)
```

The output confirms 34,500 orders, 1,903 returns (5.52%), Fashion as the highest-return category (8.28%), and “Not as described” as the most frequent recorded reason (490 returns). Mean recorded shipping cost was 6.63 for returned orders versus 6.12 for non-returned orders; mean product price was 145.12 versus 117.89, respectively.

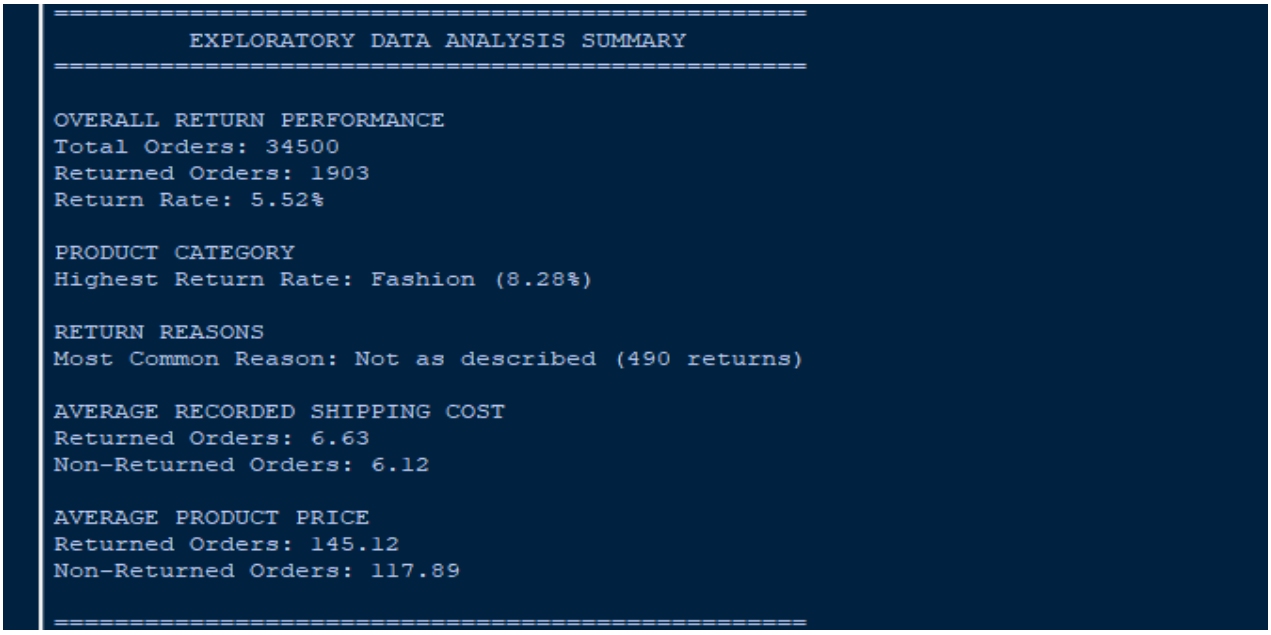


Figure 2.6 — Consolidated EDA summary output from Python

2.10 Consolidated Python Visualizations

The script includes Matplotlib bar charts for category return rates, return reasons, regional return rates, average shipping cost by return status, price-band return rates, and delivery-group return.

```
# COMBINED EDA VISUALIZATIONS
# ------------------------------------------

fig, axes = plt.subplots(2, 2, figsize=(14, 9))

# 1. Return Rate by Product Category
category_return_rate = (
    df.groupby('category')['returned']
    .apply(lambda x: (x == 'Yes').mean() * 100)
    .sort_values(ascending=False)
)

category_return_rate.plot(
    kind='bar', ax=axes[0, 0], color='steelblue'
)
axes[0, 0].set_title('Return Rate by Product Category')
axes[0, 0].set_xlabel('Product Category')
axes[0, 0].set_ylabel('Return Rate (%)')
axes[0, 0].tick_params(axis='x', rotation=35)


# 2. Major Reasons for Product Returns
return_reasons = (
    df[df['returned'] == 'Yes']['return_reason']
    .value_counts()
)

return_reasons.plot(
    kind='bar', ax=axes[0, 1], color='cornflowerblue'
)
axes[0, 1].set_title('Major Reasons for Product Returns')
axes[0, 1].set_xlabel('Return Reason')
axes[0, 1].set_ylabel('Number of Returns')
axes[0, 1].tick_params(axis='x', rotation=35)
```

```
# 3. Return Rate by Region
region_return_rate = (
    df.groupby('region')['returned']
    .apply(lambda x: (x == 'Yes').mean() * 100)
    .sort_values(ascending=False)
)

region_return_rate.plot(
    kind='bar', ax=axes[1, 0], color='royalblue'
)
axes[1, 0].set_title('Return Rate by Region')
axes[1, 0].set_xlabel('Region')
axes[1, 0].set_ylabel('Return Rate (%)')
axes[1, 0].tick_params(axis='x', rotation=0)


# 4. Average Shipping Cost by Return Status
shipping_cost_status = (
    df.groupby('returned')['shipping_cost'].mean()
)

shipping_cost_status = shipping_cost_status.reindex(['No', 'Yes'])

shipping_cost_status.plot(
    kind='bar', ax=axes[1, 1], color='skyblue'
)
axes[1, 1].set_title('Average Shipping Cost by Return Status')
axes[1, 1].set_xlabel('Return Status')
axes[1, 1].set_ylabel('Average Recorded Shipping Cost')
axes[1, 1].tick_params(axis='x', rotation=0)

# Final Layout
fig.suptitle(
    'E-Commerce Returns - Exploratory Data Analysis',
    fontsize=16, fontweight='bold'
)
plt.tight_layout(rect=[0, 0, 1, 0.95])
plt.show()
```

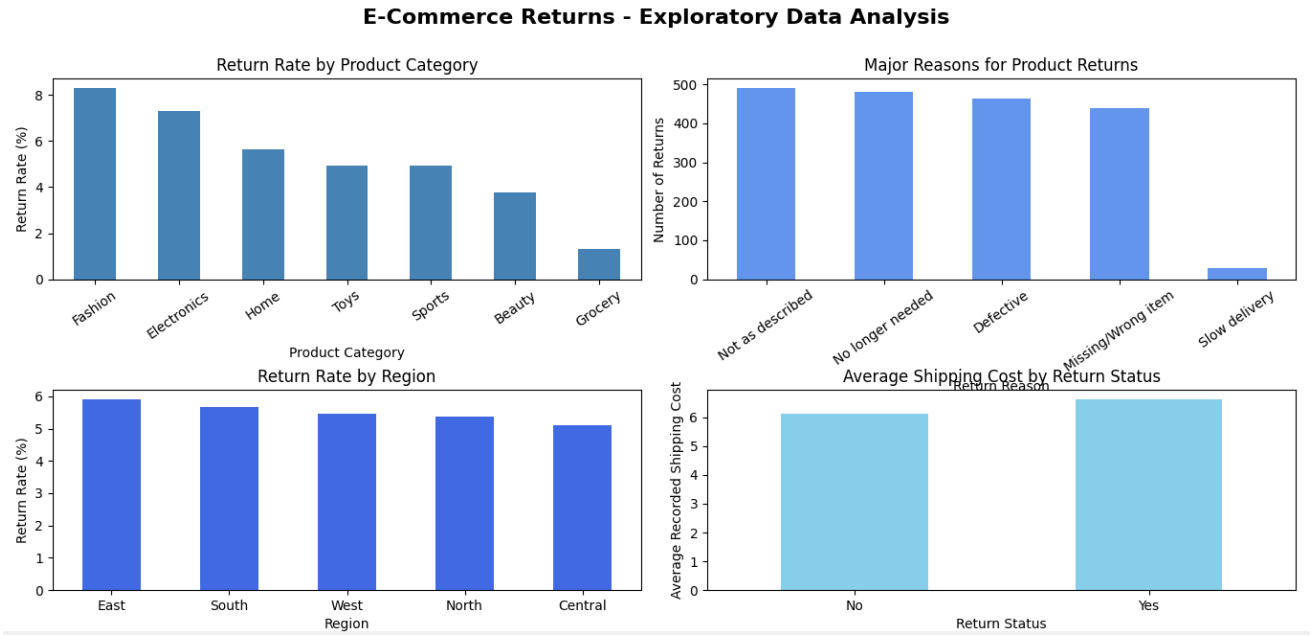


Figure 2.7 — Combined EDA visualizations: category returns, return reasons, regional returns and average recorded shipping cost

3.MYSQL: DATABASE MANAGEMENT AND SQL ANALYSIS

3.1 Introduction

MySQL was used to store the cleaned e-commerce dataset and perform structured analytical queries. SQL aggregations supported validation of Python findings and examination of returns by category, reason, region, price and cost-related indicators. The analysis used the ecommerce_returns database and ecommerce_data table.

3.2 Database Connection and Data Import

The cleaned CSV was loaded into MySQL using Python and mysql.connector. The script connected to a local MySQL server, selected the ecommerce_returns database, and inserted the 22-column dataset into ecommerce_data. Missing Pandas values were converted to Python None so they could be represented as SQL NULL. The import used executemany() in batches of 1,000 rows, with a commit after each batch.

```
cursor.execute("USE ecommerce_returns")
df_mysql = df.astype(object).where(pd.notnull(df), None)
data = list(df_mysql.itertuples(index=False, name=None))
for i in range(0, len(data), 1000):
    cursor.executemany(insert_query, data[i:i+1000])
    mydb.commit()
```

3.3 Data Verification

```
SELECT COUNT(*) AS total_records
FROM ecommerce_data;
```



Figure 3.1 — Verification of 34,500 imported records

The record count confirmed that the MySQL table contained 34,500 rows, matching the Python dataset.

3.4 Overall Return Performance

```
SELECT COUNT(*) AS total_orders,
 SUM(CASE WHEN returned = 'Yes' THEN 1 ELSE 0 END) AS returned_orders,
 ROUND(100.0 * SUM(CASE WHEN returned = 'Yes' THEN 1 ELSE 0 END) / COUNT(*), 2) AS return_rate
FROM ecommerce_data;
```

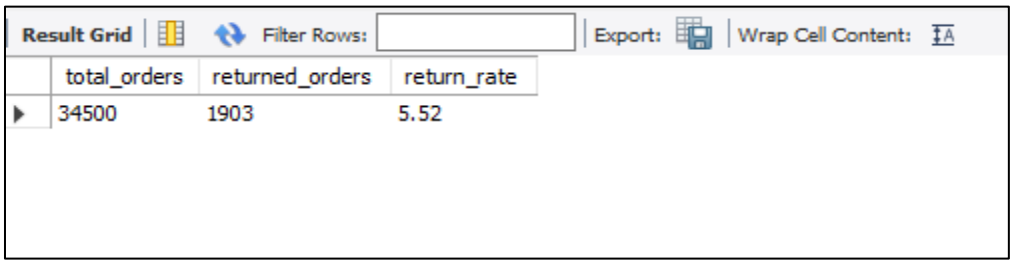
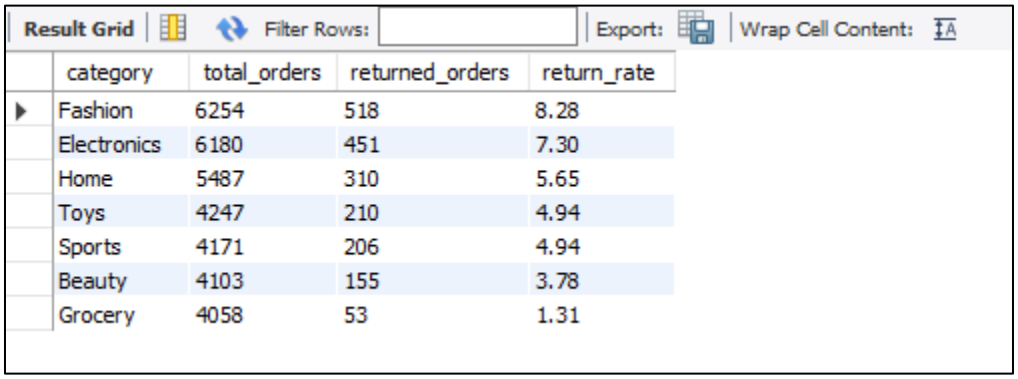


Figure 3.2 — Overall return performance

The analysis identified 1,903 returns out of 34,500 orders, producing an overall return rate of 5.52%. These figures agree with the Python EDA.

3.5 Category-Wise Return Analysis

The category query grouped records by category, counted orders and returned orders using COUNT() and SUM(CASE WHEN), and ranked categories by return rate.



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category	total_orders	returned_orders	return_rate
Fashion	6254	518	8.28
Electronics	6180	451	7.30
Home	5487	310	5.65
Toys	4247	210	4.94
Sports	4171	206	4.94
Beauty	4103	155	3.78
Grocery	4058	53	1.31

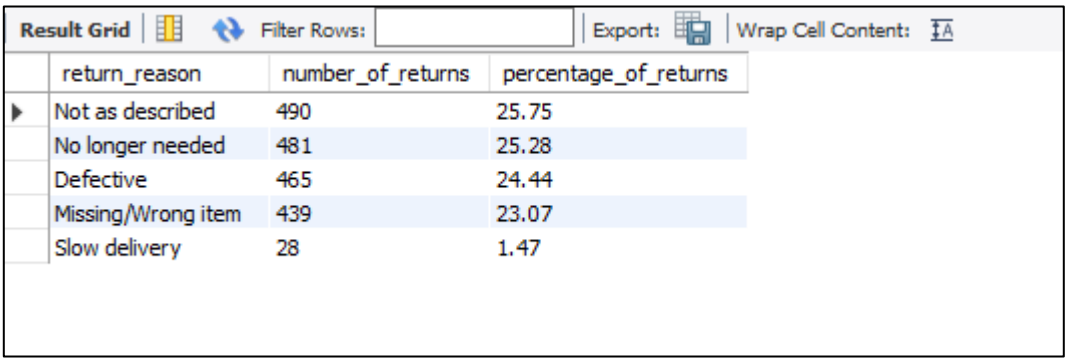
Figure 3.3 — Category-wise return analysis

Fashion had the highest return rate (8.28%; 518 of 6,254 orders), followed by Electronics (7.30%; 451 of 6,180). Grocery had the lowest return rate (1.31%). These patterns help prioritize categories for further investigation.

3.6 Return Reasons Analysis

```
SELECT return_reason, COUNT(*) AS number_of_returns,
 ROUND(100.0 * COUNT(*) /
   (SELECT COUNT(*) FROM ecommerce_data WHERE returned = 'Yes'), 2) AS percentage_of_returns
FROM ecommerce_data
WHERE returned = 'Yes'
GROUP BY return_reason

ORDER BY number_of_returns DESC;
```



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

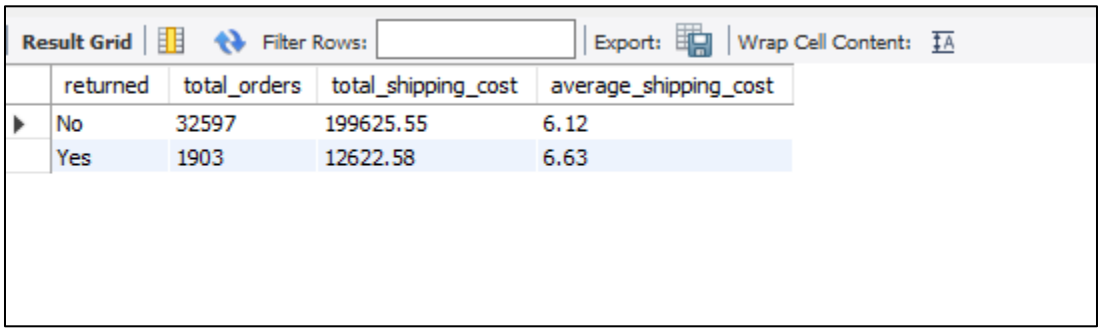
return_reason	number_of_returns	percentage_of_returns
Not as described	490	25.75
No longer needed	481	25.28
Defective	465	24.44
Missing/Wrong item	439	23.07
Slow delivery	28	1.47

Figure 3.4 — Valid return reasons, excluding non-returned orders

The original SQL file grouped all orders, producing a NULL reason row for 32,597 non-returned orders. For accurate reporting, the query was corrected by filtering returned = Yes. “Not as described” was the most frequent reason (490, 25.75%), followed by “No longer needed” (481, 25.28%), “Defective” (465, 24.44%), “Missing/Wrong item” (439, 23.07%) and “Slow delivery” (28, 1.47%).

3.7 Shipping Cost Analysis

```
SELECT returned, COUNT(*) AS total_orders,
 ROUND(SUM(shipping_cost), 2) AS total_shipping_cost,
 ROUND(AVG(shipping_cost), 2) AS average_shipping_cost
FROM ecommerce_data GROUP BY returned;
```

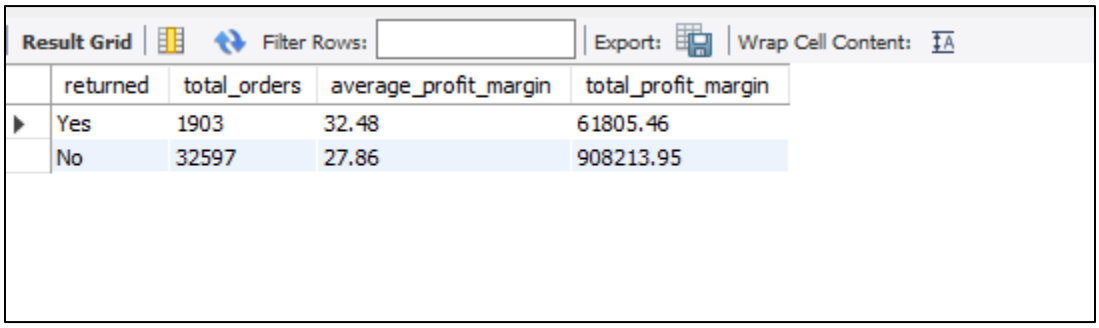


returned	total_orders	total_shipping_cost	average_shipping_cost
No	32597	199625.55	6.12
Yes	1903	12622.58	6.63

Figure 3.5 — Recorded shipping cost by return status

Returned orders had an average recorded shipping cost of 6.63, versus 6.12 for non-returned orders, a difference of 0.51 (approximately 8.3%). This is an association in the recorded outbound shipping-cost field, not a measure of return transport, handling, or other reverse-logistics expenses. No currency is assumed.

3.8 Profit-Margin Analysis



returned	total_orders	average_profit_margin	total_profit_margin
Yes	1903	32.48	61805.46
No	32597	27.86	908213.95

Figure 3.6 — Recorded profit-margin values by return status

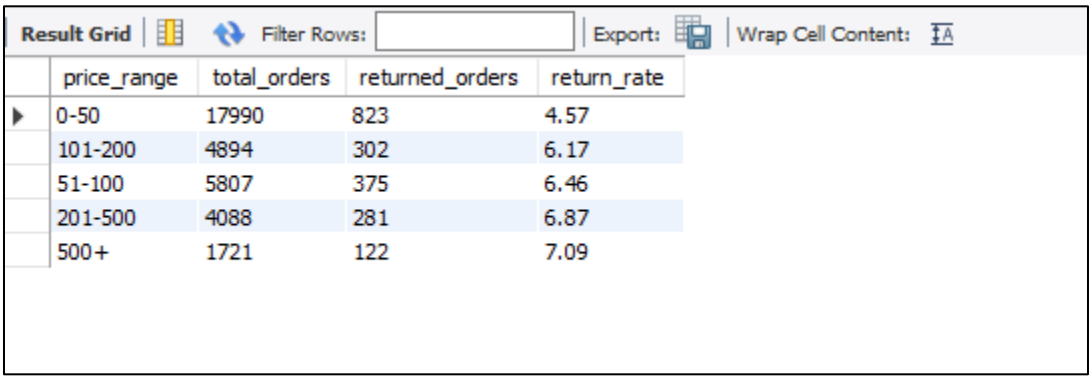
The average recorded profit_margin value was 32.48 for returned orders and 27.86 for non-returned orders. The SQL query also calculated sums of this field.

3.9 Product Price and Return Behavior

The price-range query used CASE to classify product prices into five groups and calculated the return rate within each group. The separate average-price query showed an average price of 145.12 for returned orders and 117.89 for non-returned orders.

```
SELECT CASE WHEN price <= 50 THEN '0-50'
 WHEN price <= 100 THEN '51-100'
 WHEN price <= 200 THEN '101-200'
 WHEN price <= 500 THEN '201-500' ELSE '500+' END AS price_range,
```

```
    COUNT(*) AS total_orders,
    SUM(CASE WHEN returned = 'Yes' THEN 1 ELSE 0 END) AS returned_orders,
    ROUND(100.0 * SUM(CASE WHEN returned = 'Yes' THEN 1 ELSE 0 END) / COUNT(*), 2) AS return_rate
FROM ecommerce_data GROUP BY price_range ORDER BY return_rate;
```



price_range	total_orders	returned_orders	return_rate
0-50	17990	823	4.57
101-200	4894	302	6.17
51-100	5807	375	6.46
201-500	4088	281	6.87
500+	1721	122	7.09

Figure 3.7 — Return rates across product price ranges

The 0–50 price range recorded a return rate of 4.57%, compared with 7.09% for 500+. Intermediate ranges were not strictly increasing, so the finding indicates an association rather than proof that price causes returns.

3.10 Additional SQL Analyses

Analysis	Finding / role
Region	East had the highest return rate (5.91%); Central the lowest (5.10%).
Delivery duration	Return rates were similar across common delivery groups; 11+ days contained only two orders.
Discount and quantity	Queries examined return patterns; no strong relationship was established.
Customer gender	Query compared return rates across recorded gender groups.
Category-level cost/profit	Query compared average recorded shipping_cost and profit_margin by category.
Return reasons by category	Query grouped returned orders by category and reported reason.

These supporting analyses are documented in the original SQL script. The main chapter prioritizes queries most relevant to the project objectives instead of reproducing every result grid.

3.11 Key Findings and Summary

SQL analysis verified the 34,500-record dataset and reproduced the overall 5.52% return rate. Fashion and Electronics were the highest-return categories, “Not as described” was the leading recorded reason, and returned orders showed higher average recorded shipping cost and product

price. MySQL enabled consistent aggregation and filtering of the cleaned data before dashboard development in Power BI.

Limitations: the dataset is synthetic; the recorded shipping_cost and profit_margin fields do not quantify reverse-shipping, refunds, inspection, restocking, or verified cost savings. Recommendations should therefore be treated as potential improvement opportunities, not proven causal or financial effects.

4. POWER BI DASHBOARD

4.1 Introduction to Power BI

Microsoft Power BI Desktop was used to develop an interactive, four-page dashboard for examining e-commerce return performance. The dashboard combines key performance indicators (KPIs), category and regional comparisons, return-reason patterns, and recorded shipping-cost and profit-margin summaries. The source is a synthetic e-commerce dataset; therefore, findings describe the available records rather than verified real-world business outcomes.

4.2 Data Loading

The previously cleaned file, ecommerce_cleaned.csv, was loaded into Power BI Desktop using Get Data → Text/CSV. The imported table was named ecommerce_cleaned and contained 34,500 order records and 22 columns. The Table view was used to confirm the imported fields and record count. The principal data cleaning and feature engineering had already been performed in Python.

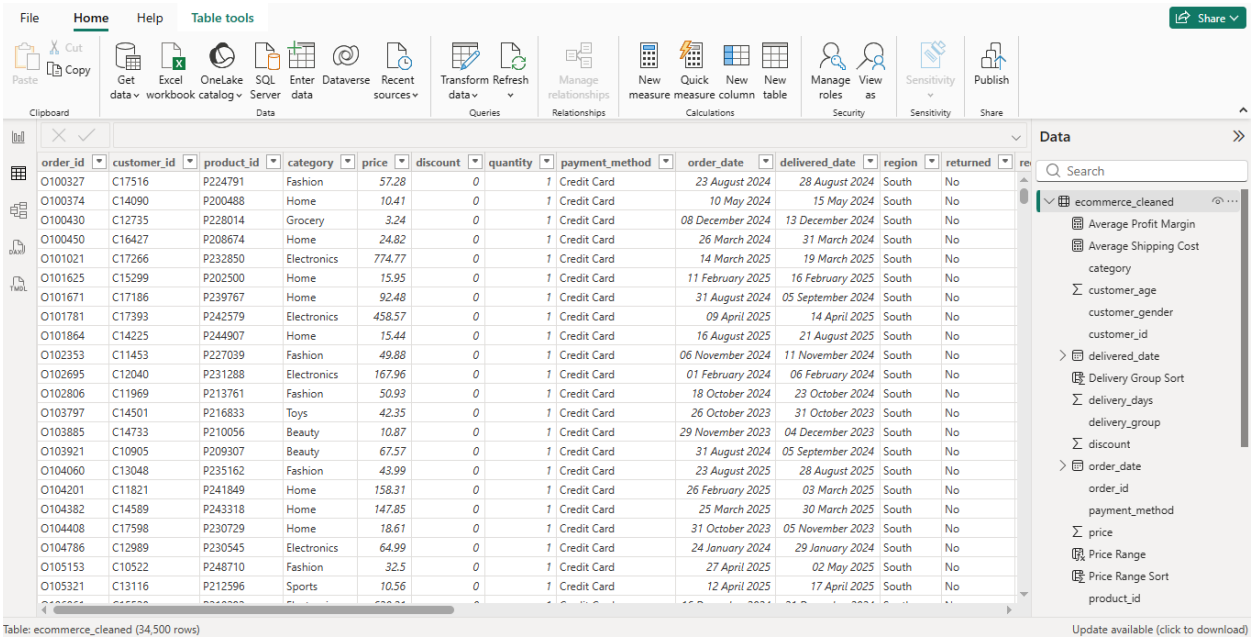


Figure 4.1 — Imported E-Commerce Dataset in Power BI Desktop

4.3 Data Verification and Preparation

Power Query Editor was used to inspect the imported dataset. Its Applied Steps pane displayed Source, Promoted Headers, and Changed Type. The screenshot confirms that identifier and category fields were treated as text and price-related fields as decimal numbers. No additional missing-value removal or duplicate deletion is claimed at this stage; those checks were addressed in the Python workflow.

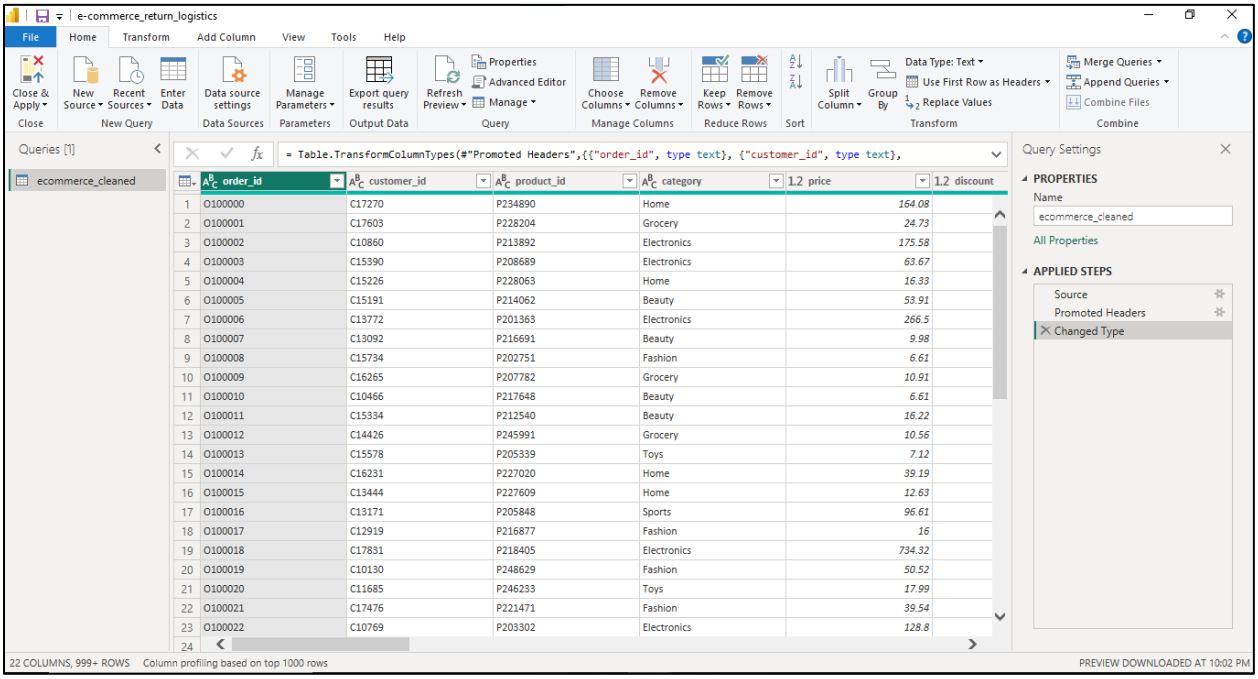


Figure 4.2 — Power Query Data Preparation and Data-Type Verification

4.4 DAX Measures

Data Analysis Expressions (DAX) measures were created to calculate dashboard KPIs. These measures evaluate in the current filter context, allowing the KPI cards and charts to respond to applicable slicer selections.

```
Total Orders = COUNTROWS(ecommerce_cleaned)

Returned Orders =
CALCULATE(COUNTROWS(ecommerce_cleaned),
        ecommerce_cleaned[returned] = "Yes")

Return Rate = DIVIDE([Returned Orders], [Total Orders], 0)

Average Shipping Cost = AVERAGE(ecommerce_cleaned[shipping_cost])

Average Profit Margin = AVERAGE(ecommerce_cleaned[profit_margin])
```

The Return Rate measure was formatted as a percentage with two decimal places. Average Shipping Cost and Average Profit Margin summarize existing dataset fields; they are not measures of full reverse-logistics expenditure or post-refund net profit.

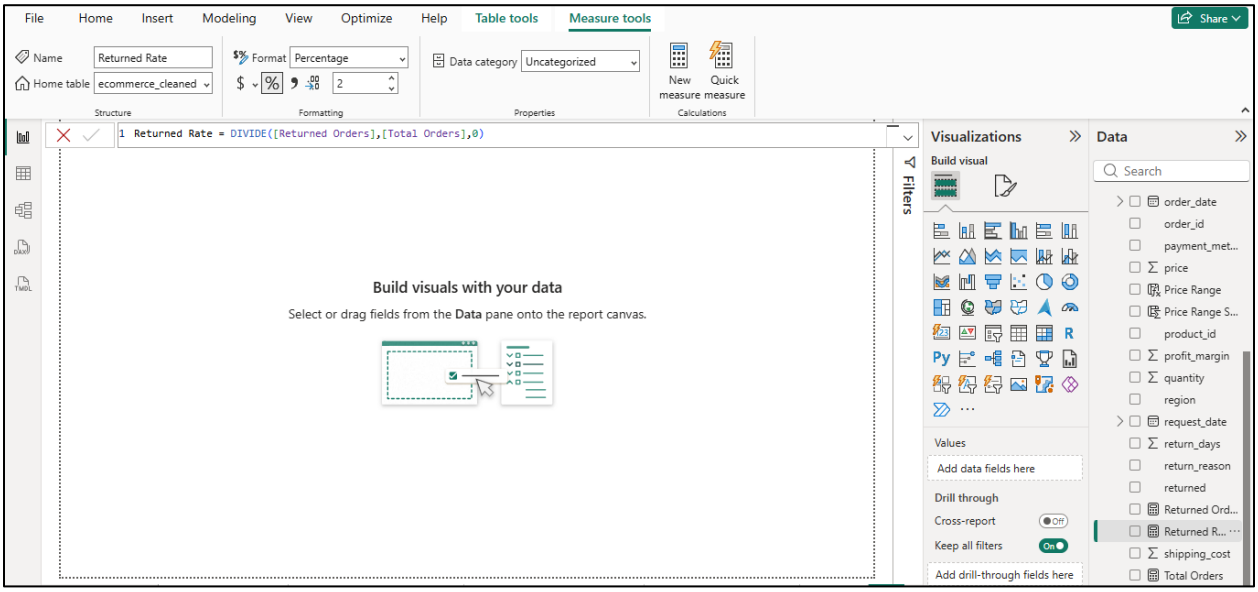


Figure 4.3 — Example DAX Measure: Return Rate

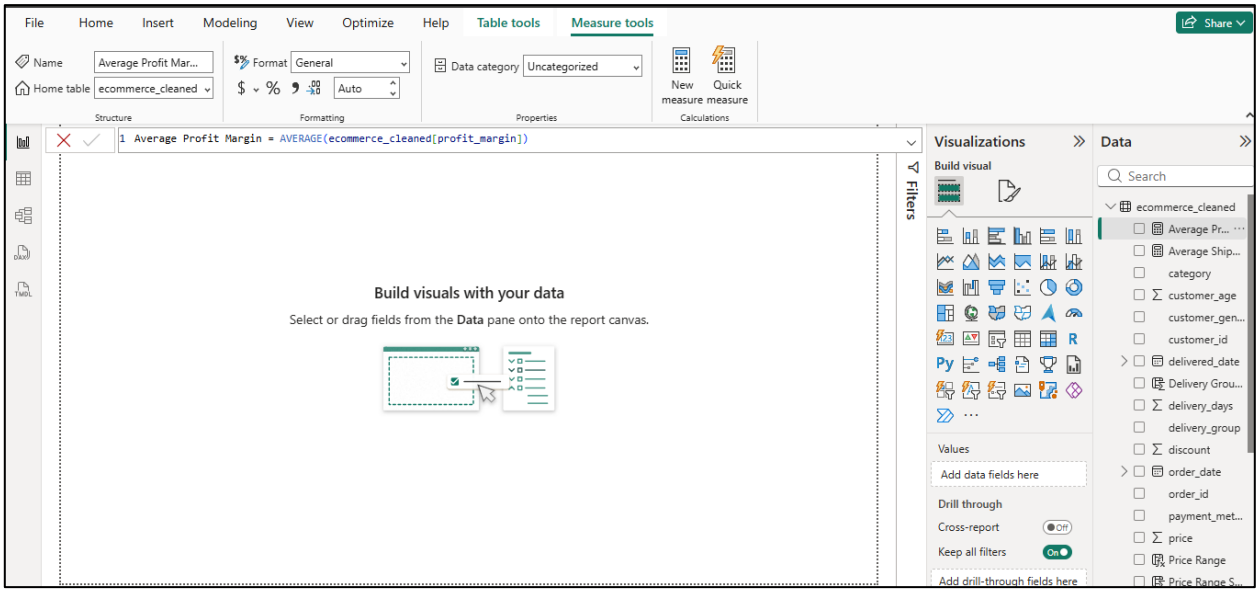


Figure 4.4 — Example DAX Measure: Average Profit Margin

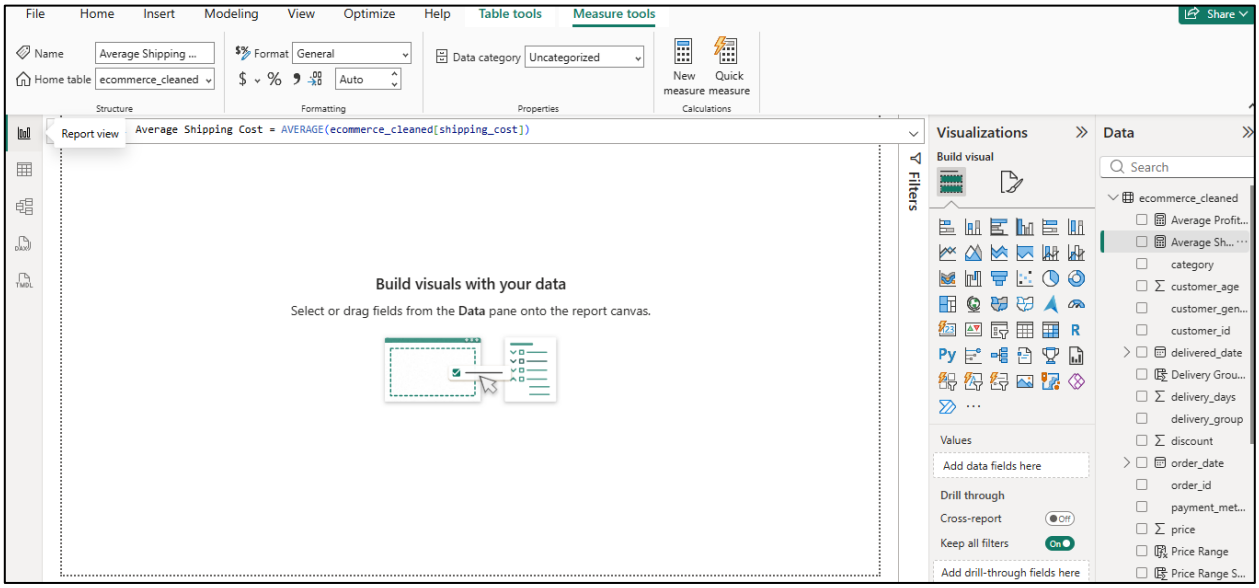


Figure 4.5 — Example DAX Measure: Average Shipping Cost

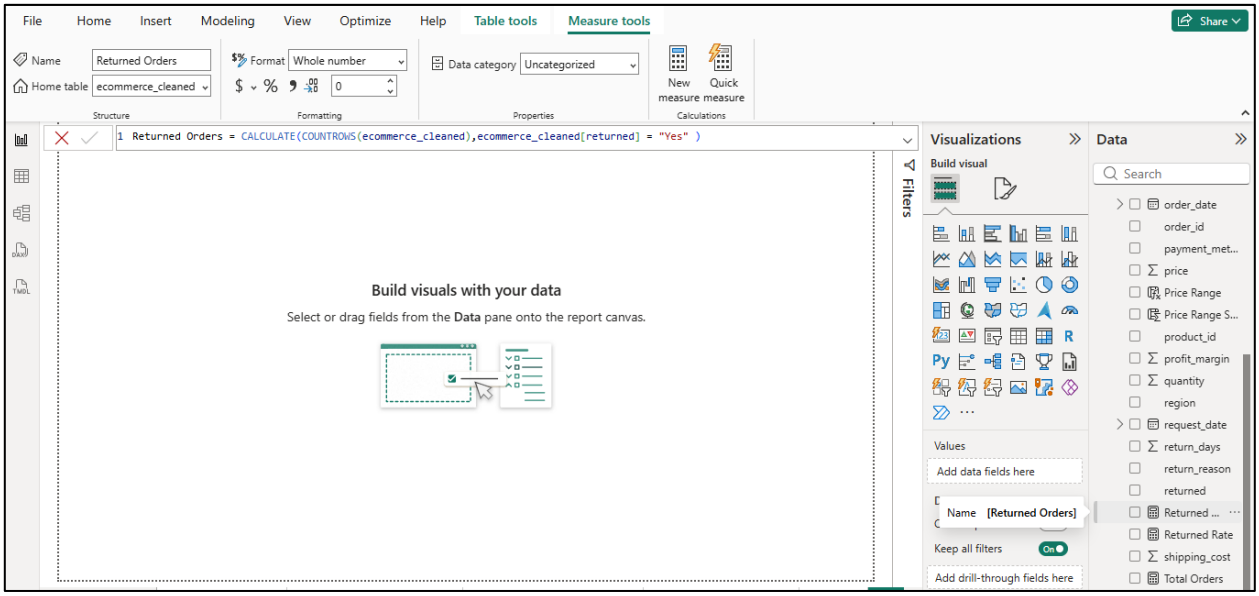


Figure 4.6 — Example DAX Measure: Returned Order

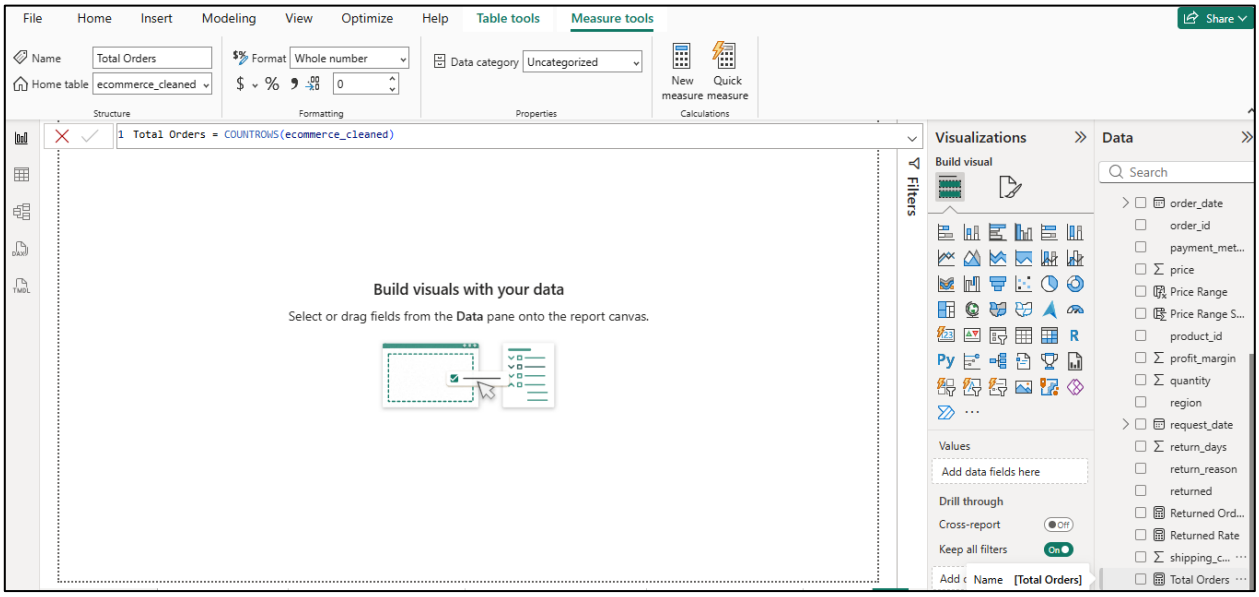


Figure 4.7 — Example DAX Measure: Total Order

4.5 Calculated Columns

A Price Range calculated column used SWITCH(TRUE()) to group product prices into 0–50, 51–100, 101–200, 201–500 and 500+. Additional Price Range Sort and Delivery Group Sort columns supported logical chart ordering instead of alphabetical sorting. The original delivery_group field was generated during Python preparation.

```
Price Range =
SWITCH(
    TRUE(),
    ecommerce_cleaned[price] <= 50, "0-50",
    ecommerce_cleaned[price] <= 100, "51-100",
    ecommerce_cleaned[price] <= 200, "101-200",
    ecommerce_cleaned[price] <= 500, "201-500",
    "500+"
)
```

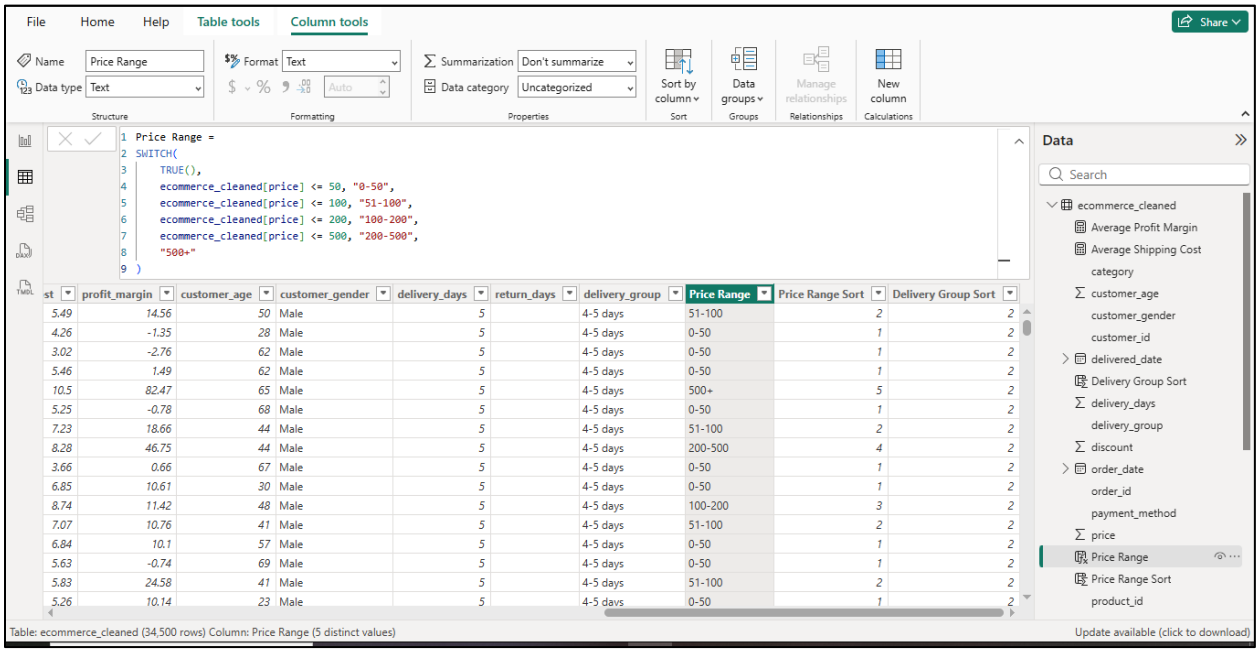


Figure 4.8 — Price Range Calculated Column Using DAX

4.6 Executive Overview Dashboard

The first report page presents five KPI cards: Total Orders, Returned Orders, Return Rate, Average Shipping Cost, and Average Profit Margin. With the full date range and no category or region restriction, the page shows 34,500 orders, 1,903 returned orders, and a 5.52% return rate. Average recorded shipping cost is 6.15 and average recorded profit-margin value is 28.12.

Horizontal bar charts compare return rates by region and product category. East has the highest regional return rate (5.91%), while Fashion has the highest category return rate (8.28%). Category, Region, and Order Date slicers allow interactive exploration.

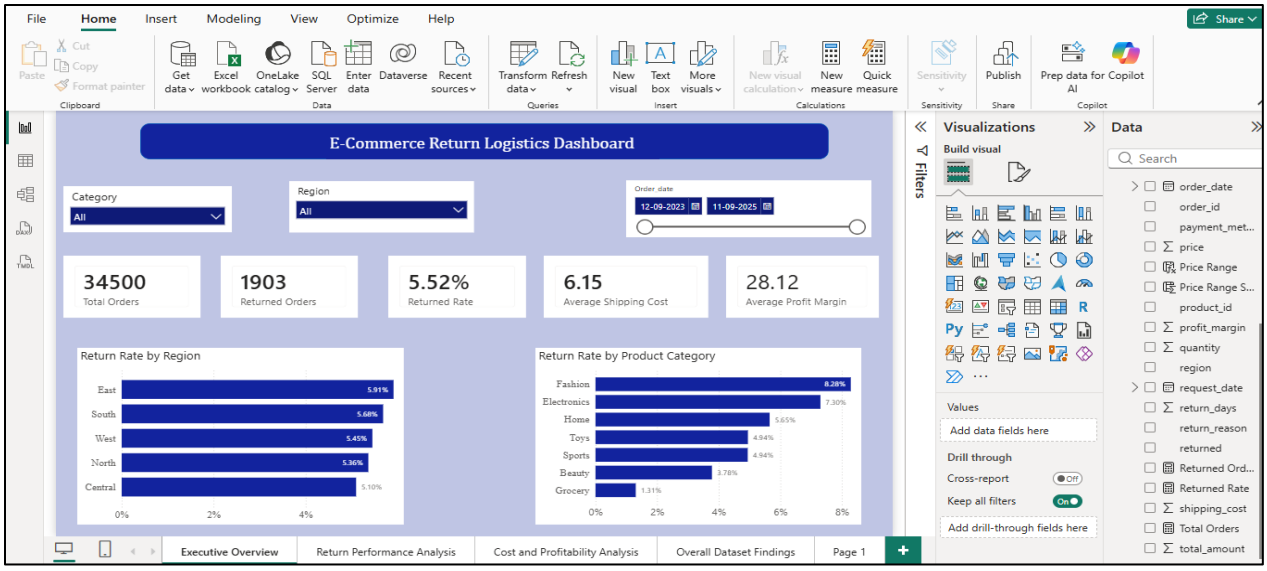


Figure 4.9 — Executive Overview Dashboard

4.7 Return Performance Analysis Dashboard

The second page contains a donut chart of return reasons, a stacked bar chart of return reasons by category, a column chart of return rate by price range, and a line chart of return rate by delivery-time group. “Not as described” is the most common recorded reason, with 490 returns (25.75%). The 0–50 price group has a 4.57% return rate, compared with 7.09% for the 500+ group. These are observed associations, not evidence of causation.

The delivery-time chart displays the more populated groups through 8–10 days; the 11+ days group contained only two orders and was omitted from that visual. Category, Region and Order Date slicers support filtered comparisons.

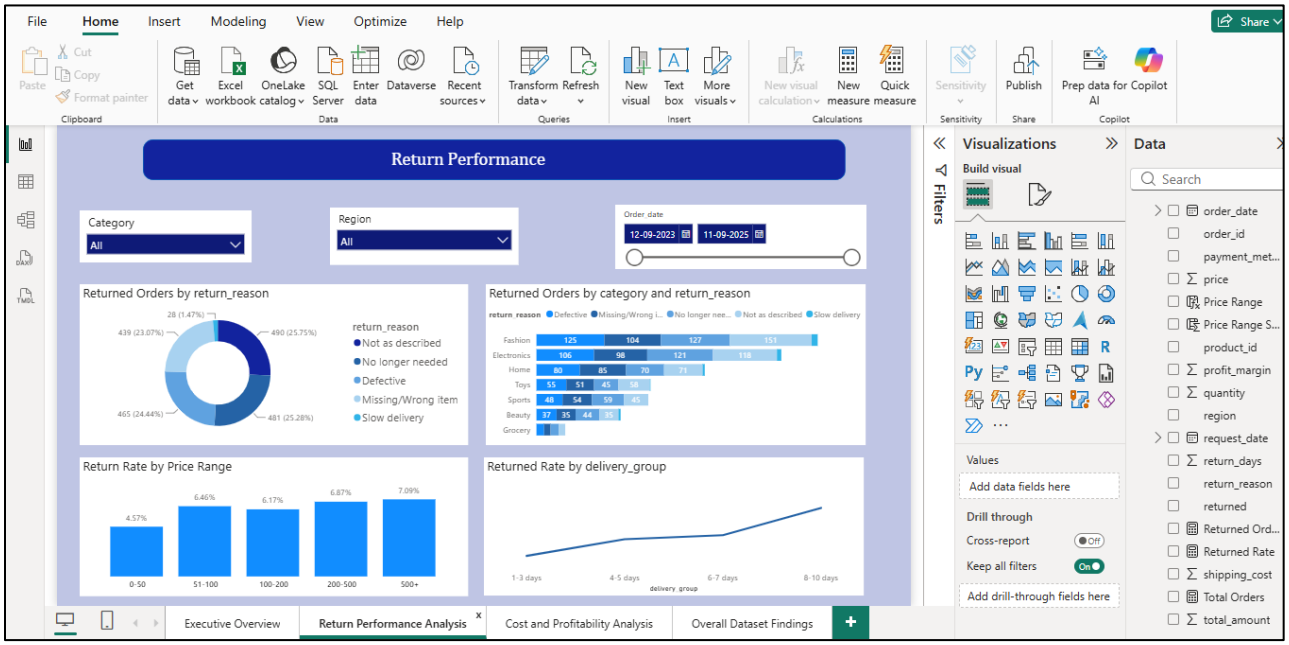


Figure 4.10 — Return Performance Analysis Dashboard

4.8 Cost and Profitability Analysis Dashboard

The third page uses four charts to compare average recorded shipping cost by return status and product category, and average recorded profit-margin values by product category and return status. Returned orders have average recorded shipping cost of 6.63 versus 6.12 for non-returned orders. Electronics has the highest average recorded shipping cost among categories (approximately 8.7).

The average recorded profit-margin value is 32.48 for returned orders and 27.86 for non-returned orders. The category chart shows Electronics as highest (55.72) and Grocery as negative (−2.26).

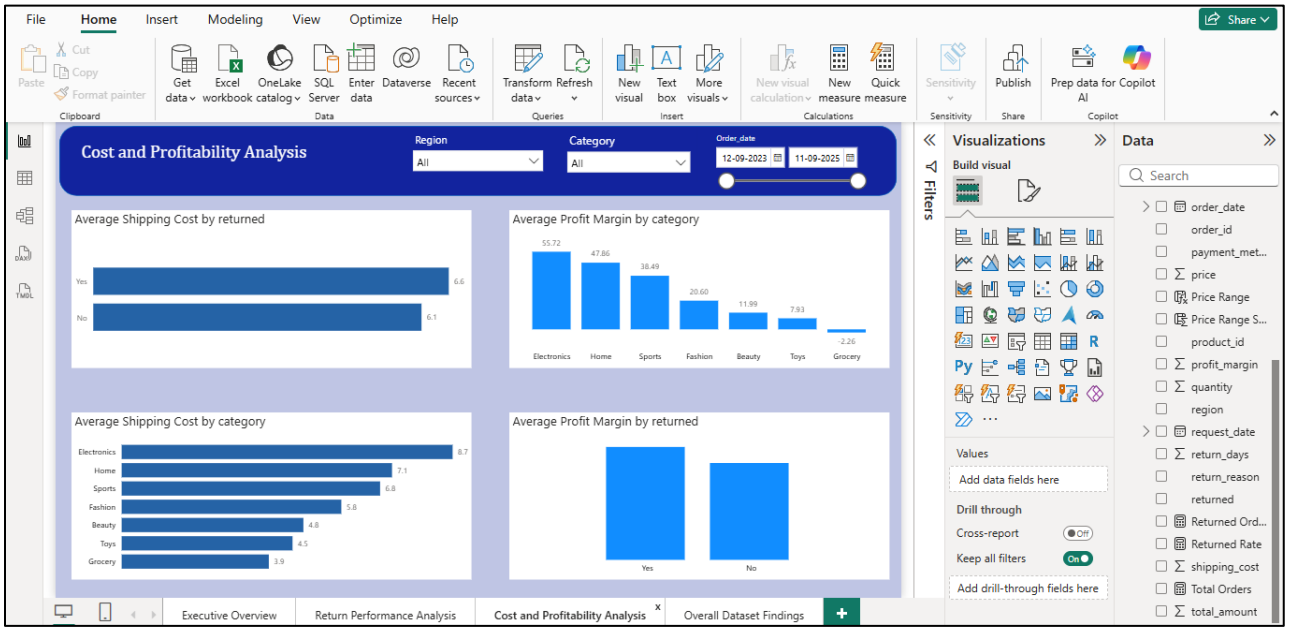


Figure 4.11---Cost and Profitability Analysis Dashboard

4.9 Key Findings and Recommendations

The final page summarizes four opportunities identified from the analysis. First, Fashion (8.28%) and Electronics (7.30%) warrant investigation of product descriptions, size guidance, and quality checks. Second, the 490 “Not as described” returns support reviewing product information accuracy. Third, the difference between average recorded shipping costs of returned and non-returned orders (6.63 versus 6.12) motivates reviewing packaging and delivery efficiency. Fourth, the return-rate comparison between 0–50 (4.57%) and 500+ (7.09%) price groups suggests investigating expectations and quality among higher-priced products.

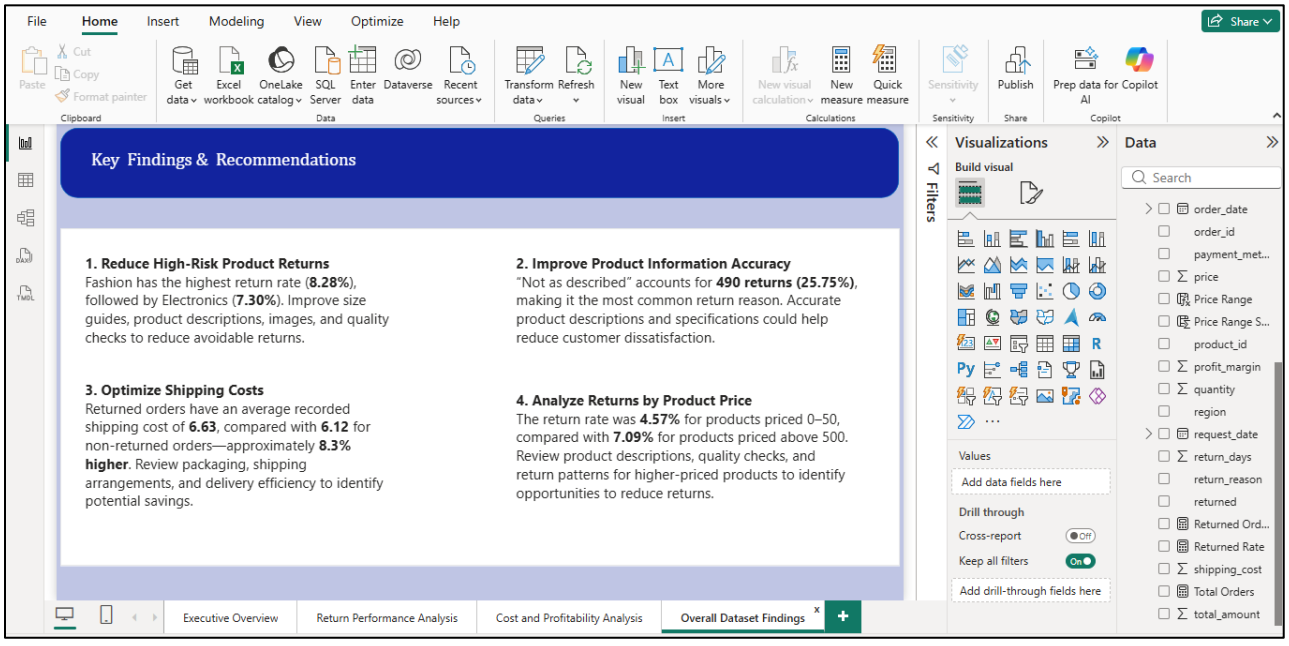


Figure 4.12 — Key Findings and Recommendations Dashboard

4.10 Dashboard Interactivity and Limitations

Slicers for category, region, and order date on the analytical pages enable filtered exploration. DAX measures recalculate within the selected context. The four-page structure separates overall KPIs, return drivers, recorded financial fields, and actionable recommendations for easier navigation.

The dataset is synthetic and lacks distinct reverse-shipping, refund, inspection, restocking, disposal, and return-handling cost fields. Therefore, this dashboard identifies return patterns and potential cost-optimization areas but does not calculate actual reverse-logistics cost or verified savings. Recorded profit_margin values also cannot establish realized profit after returns.

4.11 Chapter Summary

Power BI was used to transform the Python-cleaned dataset into an interactive report with DAX-based KPIs, price-range classifications, and four analytical pages. The results highlight Fashion and Electronics return rates, the prevalence of “Not as described” returns, and differences in recorded shipping costs and product-price groups. These findings provide a basis for further operational investigation while remaining subject to the dataset’s limitations.

5. EXECUTIVE RECOMMENDATIONS

5.1 Improve Product Information and Accuracy

“Not as described” was the most frequently reported return reason, representing 25.75% of returned orders. Businesses could improve product descriptions, photographs, specifications, and size guides to help customers make informed decisions and potentially reduce expectation-related returns.

5.2 Prioritize High-Return Product Categories

Fashion had the highest return rate (8.28%), followed by Electronics (7.30%). Category-specific reviews of sizing, product quality, customer feedback, and technical specifications may help identify avoidable return causes.

5.3 Strengthen Quality Control and Order Accuracy

Defective products and missing or incorrect items were among the major reported reasons for returns. Pre-dispatch quality checks, accurate picking and packing, and order verification could help address these issues.

5.4 Review Shipping Efficiency and Cost Management

Returned orders had a higher average recorded shipping-cost value (6.63) than non-returned orders (6.12). Businesses could review packaging, delivery processes, and logistics-provider performance. These recorded values do not measure return transportation or other reverse-logistics expenses.

6. LIMITATIONS AND FUTURE SCOPE

6.2 Limitations of the Project

Synthetic dataset: The 34,500 order records are synthetic and may not fully represent real customer behavior or operational conditions.

Missing reverse-logistics costs: The dataset does not contain actual return transportation, inspection, handling, restocking, refund-processing, or disposal expenses. Total reverse-logistics costs and verified savings cannot be calculated.

Descriptive analysis: Observed relationships between return status and factors such as price, delivery duration, and shipping cost do not establish causation.

No predictive modeling: Machine-learning models were not implemented; the project analyzes recorded return patterns rather than predicting individual future returns.

6.3 Future Scope

Machine-learning-based return prediction: Models could estimate return likelihood using information available before a return, such as product category, price, region, and shipping method. Evaluation should account for the low proportion of returned orders and use real-world validation.

Detailed reverse-logistics cost analysis: Collecting return shipping, inspection, restocking, refund-processing, and recovery-value data could support accurate cost assessment.

Real-time dashboard integration: Connecting Power BI to operational systems could enable ongoing monitoring of orders, returns, and logistics indicators.

Customer behavior analysis: Purchase history, feedback, and return history could provide additional insights, with appropriate privacy safeguards.

Advanced logistics analysis: Courier performance, warehouse location, delivery distance, and routes could be studied to investigate operational efficiency.

7. CONCLUSION

The project “E-Commerce Return Logistics Performance and Cost Optimization” analyzed product return patterns, factors associated with returns, and potential opportunities for improving logistics efficiency and cost management.

A synthetic dataset of 34,500 e-commerce orders was prepared and explored using Python. MySQL supported structured analysis through SQL queries, while Power BI provided interactive visualizations of key performance indicators and return-related trends.

The overall return rate was 5.52%. Fashion and Electronics recorded the highest category-wise return rates. Product description mismatches, changing customer needs, defective products, and incorrect or missing items were major reported reasons for returns. Differences were also observed across regions and product price ranges.

Returned orders had a higher average recorded shipping-cost value than non-returned orders. The findings suggest potential actions involving clearer product information, stronger quality checks, better order accuracy, and reviews of shipping operations. The dashboards made these patterns easier to compare and communicate.

However, the synthetic dataset lacks actual reverse-logistics expenses and sufficiently defined financial measures. Therefore, the project does not calculate verified savings or establish causal relationships. Its recommendations would require validation using real business data.

Overall, the project demonstrates an end-to-end data analytics workflow using Python, MySQL, and Power BI to support informed decisions about e-commerce returns. Future work could extend the analysis with real operational data, machine-learning-based prediction, detailed cost records, and real-time monitoring.

APPENDIX

A. Project File Structure

The project consists of the original Kaggle e-commerce returns dataset, a Python notebook for cleaning and exploratory analysis, the cleaned file ecommerce_cleaned.csv (34,500 records and 22 columns), the MySQL database ecommerce_returns (table: ecommerce_data), and a four-page Power BI report.

B. Power BI Dashboard File

Dashboard: E-Commerce Return Logistics Performance and Cost Optimization. Tool: Microsoft Power BI Desktop. Pages: Executive Overview; Return Performance Analysis; Cost and Profitability Analysis; Key Findings and Recommendations.

C. Tools and Libraries Used

Tool / Library	Purpose
Python	Data cleaning and exploratory analysis
Pandas	Data manipulation and preprocessing
NumPy	Numerical operations
Matplotlib / Seaborn	Data visualization
MySQL	Database storage and SQL analysis
Power BI Desktop	Interactive dashboard development
DAX	Calculated measures and columns
Kaggle dataset	Source data

D. Dataset Summary

Description	Details
Dataset type	Synthetic e-commerce orders
Total records	34,500
Original columns	19
Cleaned columns	22
Returned orders	1,903
Overall return rate	5.52%